hf-openai-privacy-filter

---

license: apache-2.0 pipeline_tag: token-classification library_name: transformers tags:

- transformers.js

OpenAI Privacy Filter

OpenAI Privacy Filter is a bidirectional token-classification model for personally identifiable information (PII) detection and masking in text. It is intended for high-throughput data sanitization workflows where teams need a model that they can run on-premises that is fast, context-aware, and tunable.

OpenAI Privacy Filter is pretrained autoregressively to arrive at a checkpoint with similar architecture to gpt-oss, albeit of a smaller size. We then converted that checkpoint into a bidirectional token classifier over a privacy label taxonomy, and post-trained with a supervised classification loss. (For architecture details about gpt-oss, please see the gpt-oss model card.) Instead of generating text token-by-token, this model labels an input sequence in a single forward pass, then decodes coherent spans with a constrained Viterbi procedure. For each input token, the model predicts a probability distribution over the label taxonomy which consists of 8 output categories described below.

Highlights:

- Permissive Apache 2.0 license: ideal for experimentation, customization, and commercial deployment.
- Small size: Runs in a web browser or on a laptop – 1.5B parameters total and 50M active parameters.
- Fine-tunable: Adapt the model to specific data distributions through easy and data efficient finetuning.
- Long-context: 128,000-token context window enables processing long text with high throughput and no chunking.
- Runtime control: configure precision/recall tradeoffs and detected span lengths through preset operating points.

Usage

Transformers

1. Using the `pipeline` API:

```
from transformers import pipeline

classifier = pipeline(
    task="token-classification",
    model="openai/privacy-filter",
)
classifier("My name is Alice Smith")
```

2. Using as `AutoModelForTokenClassification` model:

```
import torch
from transformers import AutoModelForTokenClassification, AutoTokenizer
```

```python
tokenizer = AutoTokenizer.from_pretrained("openai/privacy-filter")
model = AutoModelForTokenClassification.from_pretrained("openai/privacy-filter",
        device_map="auto")

inputs = tokenizer("My name is Alice Smith", return_tensors="pt").to(model.device)

with torch.no_grad():
    outputs = model(**inputs)

predicted_token_class_ids = outputs.logits.argmax(dim=-1)
predicted_token_classes = [model.config.id2label[token_id.item()] for token_id in
        predicted_token_class_ids[0]]
print(predicted_token_classes)
```

Transformers.js

1. Using the `pipeline` API:

```js
import { pipeline } from "@huggingface/transformers";

const classifier = await pipeline(
  "token-classification", "openai/privacy-filter",
  { device: "webgpu", dtype: "q4" },
);

const input = "My name is Harry Potter and my email is harry.potter@hogwarts.edu.";
const output = await classifier(input, { aggregation_strategy: "simple" });
console.dir(output, { depth: null });
```

▶ See example output

Model Details

Model Description

Privacy Filter is a bidirectional token classification model with span decoding. It is trained in phases, beginning with autoregressive pretraining. The pretrained language model is then modified and post-trained as a bidirectional banded attention token classifier with band size 128 (effective attention window: 257 tokens including self). This means:

- The base model is an autoregressive pretrained checkpoint.
- The language-model output head is replaced with a token-classification head over privacy labels.
- Post-training is supervised token-level classification rather than next-token prediction.
- Inference applies constrained sequence decoding to produce coherent BIOES (Begin, Inside, Outside, End, Single) span labels.

Architecturally, the implementation in this repo is a pre-norm transformer encoder-style stack with:

- token embeddings
- 8 repeated transformer blocks
- grouped-query attention with rotary positional embeddings, with 14 query heads and 2 KV heads (group size = 7 queries per KV head)
- sparse mixture-of-experts feed-forward blocks with 128 experts total (top-4 routing per token)
- a final token-classification head over privacy labels (rather than natural language vocabulary tokens), with residual stream width `d_model = 640`.

Relative to iterative autoregressive approaches, this design allows all tokens to be labeled in one pass, which improves throughput. Relative to classical masked-language-model pretraining approaches, this is a post-training conversion of an autoregressive model rather than a native masked-LM setup.

Output Shape

Privacy Filter can detect 8 privacy span categories:

1. `account_number`
2. `private_address`
3. `private_email`
4. `private_person`
5. `private_phone`
6. `private_url`
7. `private_date`
8. `secret`

To perform token-classification, each non-background span category is expanded into boundary-tagged token classes: `B-<label>`, `I-<label>`, `E-<label>`, `S-<label>`, plus the background class, `O`. So the total number of token-level output classes is 33: 1 background class + 8 span labels * 4 boundary tags = 33 classes. This means the output head emits 33 logits for each token. For a sequence of length T, the output has shape `[T, 33]`; for a batch of size B, it has shape `[B, T, 33]`.

The token-label vocabulary consists of the background label `O` plus BIOES-tagged variants of each privacy category: `account_number`, `private_address`, `private_email`, `private_person`, `private_phone`, `private_url`, `private_date`, and `secret`. In other words, for each category, the model predicts `B-`, `I-`, `E-`, and `S-` forms corresponding to begin, inside, end, and single-token spans. At inference time, these per-token logits are decoded into coherent BIOES span labels using constrained sequence decoding.

Sequence Decoding Rationale and Calibration

Rationale

After the token classifier produces per-token logits, we decode labels with a constrained Viterbi decoder using linear-chain transition scoring, rather than taking an independent argmax for each token. The decoder enforces allowed BIOES boundary transitions and scores complete label paths with start, transition, and end terms, plus six transition-bias parameters that control background persistence, span entry, span continuation, span closure, and boundary-to-boundary handoff. This global path optimization is intended to improve span coherence and boundary stability by making each token decision depend on sequence-level structure, not just local logits, especially in noisy or mixed-format text where local token decisions alone can produce fragmented or inconsistent boundaries.

Operating-Point Calibration

Sequence Decoding parameters can discourage staying in background while encouraging span entry and continuation, yielding broader and more contiguous masking for improved recall, or vice versa for improved precision. At runtime, users can tune parameters that control this tradeoff.

Model Metadata

- Developed by: OpenAI
- Funded by: OpenAI

- Shared by: OpenAI
- Model type: Bidirectional token classification model for privacy span detection
- Language(s): Primarily English; selected multilingual robustness evaluation reported
- License: Apache 2.0
- Source repository: https://github.com/openai/privacy-filter
- Demo: https://huggingface.co/spaces/openai/privacy-filter
- Model card: OpenAI Privacy Filter Model Card

Bias, Risks, and Limitations

Risk: Over-reliance

Privacy Filter is a redaction and data minimization aid, not an anonymization, compliance, or a safety guarantee. Over-reliance on the tool as a blanket anonymization claim would risk missing desired privacy objectives. Privacy Filter is best used as one of multiple layers in a holistic end-to-end privacy-by-design approach.

Limitation: Static Label Policy

The model will only identify personal data spans that match the trained label taxonomy and definitions. Real-life privacy use cases are varied and complex and definitions of appropriate label policies and decision boundaries can differ. Thus model defaults may not satisfy organization-specific governance requirements without calibration/fine-tuning.

Privacy Filter does not support configuring label policies dynamically at runtime; instead changing policies requires further finetuning of the model. The native label set and associated decision boundaries may not be appropriate for every use case. For example, the model's training policy aims to prioritize personal identifiers, often preserving context that is not strongly person-linked by design; some users may want to adjust this choice.

Performance may drop on non-English text, non-Latin scripts, protected-group naming patterns, or domains that are out of distribution compared to model training.

Failure Modes

Like all models, Privacy Filter can make mistakes, such as: under-detection of uncommon personal names, regional naming conventions, initials, honorific-heavy references, or domain-specific identifiers; over-redaction of public entities, organizations, locations, or common nouns when local context is ambiguous; fragmented or shifted span boundaries in mixed-format text, long documents, or text with heavy punctuation and layout artifacts; missed secrets for novel credential formats, project-specific token patterns, or secrets split across surrounding syntax; and over-redaction of benign high-entropy strings, placeholders, hashes, sample credentials, or synthetic examples that resemble secrets.

These limitations can interact with demographic, regional, and domain variation. For example, names and identifiers that are underrepresented in training data, or that follow conventions different from the dominant training distribution, may be more likely to be missed or inconsistently bounded.

High-Risk Deployment Caution

Additional caution is warranted in high-sensitivity settings such as medical, legal, financial, human resources, education, and government workflows. In these settings, both false negatives and false positives can be costly: missed spans may expose sensitive information, while excess masking can remove material context needed for review, auditing, or downstream decision-making.

Recommendations

- Use Privacy Filter as part of a holistic privacy-by-design approach, not as a blanket anonymization claim.
- Evaluate in-domain with local policy references before production.
- Use task-specific fine-tuning when policy differs from base boundaries.
- Keep human review paths for high-sensitivity workflows.